Unit 4 - Normalization

Database Management (DI03016041) - GTU Diploma IT Sem 3

19% weightage | 6 hours

How to use these notes: every topic is written in a descriptive, copy-ready way. **3 marks** = definition + 2-3 points. **4 marks** = add example or table. **7 marks** = write everything with diagram / example. Lines marked ★ show where the topic was asked in a GTU paper (Winter 2025 / Summer 2026). SQL examples use Oracle syntax.

GTU questions already asked from this unit (do these first)

- FD asked in W2025 Q4(a) [3M: functional dependency with example]
- Asked in S2026 Q5(b) [4M: explain full functional dependency]
- Asked in W2025 Q5(a)OR [3M: transitive functional dependency]
- 1NF asked in BOTH papers: W2025 Q4(c)OR [7M: criteria + two techniques] + S2026 Q4(c) [7M: need + 1NF] + S2026 Q4(a)OR(b) [4M: convert table to 1NF]
- S2026 Q4(b)OR [4M: convert Enrollment table to 1NF]
- 2NF asked in S2026 Q4(c)OR [7M: explain 2NF with example] + W2025 Q4(c) [7M: convert Doctor_Patient to 2NF]
- W2025 Q4(c) [7M: Doctor_Patient - prime and non-prime attributes, FDs, convert to 2NF]
- Asked in W2025 Q4(a)OR [3M: explain 3NF with example]
- BCNF asked in W2025 Q5(a) [3M: differentiate 3NF vs BCNF]

4.1 Anomalies created by poor database design

Poor database design stores the same data many times in one table. This causes **anomalies** (problems). Example table **STUDENT_COURSE**:

Roll_no	Name	Course_id	Course_name	Fees
1	Ravi	C1	Java	5000
1	Ravi	C2	SQL	4000
2	Meera	C1	Java	5000

- **Insertion anomaly:** cannot add a new course until a student joins it (Roll_no is part of the key), or we must repeat data.
- **Update anomaly:** to change the name of course C1, every row containing C1 must be changed; missing one makes data inconsistent.
- **Deletion anomaly:** if Meera's only row is deleted, the information about course C1 may be lost if no other row has it.

4.2 Normalization: definition, importance and goals

Normalization is the process of **organising the data of a database into well-structured tables** by decomposing big tables into smaller ones, so that redundancy and anomalies are removed. It is done by applying rules called **normal forms** (1NF, 2NF, 3NF, BCNF).

Importance / need of normalization (S2026 Q4c)

1. **Reduces data redundancy** (same data is not stored again and again).
2. **Removes insertion, update and deletion anomalies.**
3. **Maintains data consistency and integrity.**
4. **Saves storage space** and makes the database easier to maintain.
5. Gives a **logical, clear table structure** and makes queries efficient.

Goals of normalization

- Eliminate redundant (duplicate) data.
- Ensure data dependencies are logical (data stored in the right table).
- Make the design flexible for future changes.
- Minimise anomalies while keeping all information (lossless decomposition).

4.3 Prime vs non-prime attributes and functional dependency

★ FD asked in W2025 Q4(a) [3M: functional dependency with example]

Prime and non-prime attributes

- **Prime attribute:** an attribute that is **part of some candidate key**.
- **Non-prime attribute:** an attribute that is **not part of any candidate key**.

Functional dependency (FD)

A **functional dependency X → Y** means: if two rows have the **same value of X, they must have the same value of Y**. We say 'X determines Y' (Y is functionally dependent on X); X is the **determinant**.

Example: In STUDENT(Roll_no, Name, City), **Roll_no → Name** and **Roll_no → City**, because one roll number gives exactly one name and city. But Name → Roll_no is not an FD, because two students may have the same name.

Roll_no	Name	City
101	Ravi	Surat
102	Meera	Rajkot
103	Ravi	Pune

Roll_no → Name holds (each roll no. has one name). Name → City does not hold (Ravi has Surat and Pune).

Types of functional dependency

Type	Meaning	Example
Full functional dependency	Y depends on the **whole** composite key X, and not on any proper part of X	(Roll_no, Course_id) → Marks
Partial dependency	Y depends on **only a part** of a composite key	(Roll_no, Course_id) → Name, but Roll_no → Name alone
Transitive dependency	A non-key attribute depends on another non-key attribute: X → Y and Y → Z, so X → Z	Emp_id → Dept_id, Dept_id → Dept_name, so Emp_id → Dept_name

4.3.2.2 Full functional dependency

★ Asked in S2026 Q5(b) [4M: explain full functional dependency]

A functional dependency **X → Y** is a **full functional dependency** if Y is functionally dependent on X as a whole, and **not dependent on any proper subset of X**. It matters when X is a composite (multi-attribute) key.

Example: Table MARKS(Roll_no, Course_id, Marks, Student_name). **(Roll_no, Course_id) → Marks** is a full FD: marks depend on both the student and the course; neither Roll_no alone nor Course_id alone can give the marks. But **(Roll_no, Course_id) → Student_name** is **not full** (it is a partial dependency) because Roll_no alone determines Student_name.

- Full FD: removing any attribute from the left side breaks the dependency.
- A table is in **2NF** only when all non-prime attributes are **fully** dependent on the primary key.
- If the key has a single attribute, every FD on it is automatically full.

4.3.2.3 Transitive dependency

★ Asked in W2025 Q5(a)OR [3M: transitive functional dependency]

A **transitive dependency** exists when a **non-key attribute depends on another non-key attribute** instead of depending directly on the primary key. If **A → B** and **B → C** (and B does not determine A), then **A → C** is a transitive dependency.

Example: EMPLOYEE(Emp_id, Emp_name, Dept_id, Dept_name). Emp_id → Dept_id and Dept_id → Dept_name, so Emp_id → Dept_name is transitive (Dept_name depends on Emp_id only through Dept_id).

Problem: department name is repeated for every employee of the department → redundancy and anomalies. **Solution:** remove it by converting to 3NF: EMPLOYEE(Emp_id, Emp_name, Dept_id) and DEPARTMENT(Dept_id, Dept_name).

4.4.1 First Normal Form (1NF)

★ 1NF asked in BOTH papers: W2025 Q4(c)OR [7M: criteria + two techniques] + S2026 Q4(c) [7M: need + 1NF] + S2026 Q4(a)OR(b) [4M: convert table to 1NF]

★ S2026 Q4(b)OR [4M: convert Enrollment table to 1NF]

First Normal Form (1NF): a table is in 1NF if it satisfies these **criteria**:

1. Every column contains only **atomic (indivisible, single) values** - no lists, no repeating groups, no multi-valued fields.
2. Each column holds values of **one kind (same domain)**.
3. Each column has a **unique name**; order of rows / columns does not matter.
4. Every row is **unique** (there is a primary key).

Example: converting to 1NF (W2025 Q4c OR)

Not in 1NF - the column Diseases has several values in one cell:

PatientID	PatientName	Diseases
P001	Meera	Fever, Cold
P002	Amit	Diabetes, BloodPressure, Asthma

Technique 1: separate row for each value (repeat the other data)

PatientID	PatientName	Disease
P001	Meera	Fever
P001	Meera	Cold
P002	Amit	Diabetes
P002	Amit	BloodPressure
P002	Amit	Asthma

Primary key becomes (PatientID, Disease). Simple, but PatientName is repeated.

Technique 2: decompose into two tables (separate table for the repeating group)

PATIENT (PatientID, PatientName)	PATIENT_DISEASE (PatientID, Disease)

PatientID	PatientName
P001	Meera
P002	Amit

PatientID (FK)	Disease
P001	Fever
P001	Cold
P002	Diabetes
P002	BloodPressure
P002	Asthma

No repetition of patient name; PatientID links the tables. (A third, poor method is adding columns Disease1, Disease2, Disease3 - it wastes space and limits the number of diseases, so it is not recommended.)

Given table (not in 1NF - Subjects offered has many values in one cell):

Enrollment No	Student name	Subjects offered
001	abc	os, java, dm
002	xyz	iits, Python

1NF - one value per cell (one row per subject):

Enrollment No	Student name	Subject offered
001	abc	os
001	abc	java
001	abc	dm
002	xyz	iits
002	xyz	Python

Primary key = (Enrollment No, Subject offered). Now every cell has an atomic value, so the table is in 1NF.

4.4.2 Second Normal Form (2NF)

★ 2NF asked in S2026 Q4(c)OR [7M: explain 2NF with example] + W2025 Q4(c) [7M: convert Doctor_Patient to 2NF]

★ W2025 Q4(c) [7M: Doctor_Patient - prime and non-prime attributes, FDs, convert to 2NF]

Second Normal Form (2NF): a table is in 2NF if (i) it is in **1NF** and (ii) **no non-prime attribute is partially dependent on any candidate key** - i.e. every non-key attribute is **fully functionally dependent** on the whole primary key. (2NF only matters for tables with a **composite key**.)

Example

STUDENT_MARKS(Roll_no, Course_id, Student_name, Course_name, Marks) with primary key (Roll_no, Course_id):

Roll_no	Course_id	Student_name	Course_name	Marks
1	C1	Ravi	Java	80
1	C2	Ravi	SQL	75
2	C1	Meera	Java	90

- Roll_no → Student_name (partial: depends on part of key)

- Course_id → Course_name (partial)
- (Roll_no, Course_id) → Marks (full)

These partial dependencies cause redundancy ('Ravi' and 'Java' repeated). **Decompose to 2NF** by moving each partially dependent attribute with its determinant to a new table:

STUDENT (Roll_no, Student_name)	COURSE (Course_id, Course_name)	MARKS (Roll_no, Course_id, Marks)

Now in every table, each non-key attribute depends on the **whole** key → 2NF.

Doctor_Patient(PatientID, DoctorID, PatientName, DoctorName, Department)

PatientID	DoctorID	PatientName	DoctorName	Department
P01	D01	Meera	Dr. Patel	Cardiology
P02	D02	Ankit	Dr. Shah	Neurology
P03	D01	Ramesh	Dr. Patel	Cardiology
P01	D03	Meera	Dr. Vasava	Orthopedic

a) Prime and non-prime attributes

PatientID alone repeats (P01 appears twice) and DoctorID alone repeats (D01 appears twice), so neither is a key alone. The **candidate key is (PatientID, DoctorID)**.

- **Prime attributes:** PatientID, DoctorID.
- **Non-prime attributes:** PatientName, DoctorName, Department.

b) Functional dependencies

1. PatientID → PatientName
2. DoctorID → DoctorName
3. DoctorID → Department (so DoctorID → DoctorName, Department)
4. (PatientID, DoctorID) → PatientName, DoctorName, Department (the whole key determines everything)

PatientName depends on part of the key (PatientID) and DoctorName / Department depend on part of the key (DoctorID) → these are **partial dependencies**, so the relation is **not in 2NF**.

c) Conversion to 2NF

Remove partial dependencies by splitting into three tables:

PatientID (PK)	PatientName
P01	Meera
P02	Ankit
P03	Ramesh

PATIENT(PatientID, PatientName)

DoctorID (PK)	DoctorName	Department
D01	Dr. Patel	Cardiology
D02	Dr. Shah	Neurology
D03	Dr. Vasava	Orthopedic

DOCTOR(DoctorID, DoctorName, Department)

PatientID (FK)	DoctorID (FK)
P01	D01
P02	D02
P03	D01
P01	D03

PATIENT_DOCTOR(PatientID, DoctorID) - composite primary key. All three tables are in 2NF (and also in 3NF).

4.4.3 Third Normal Form (3NF)

★ Asked in W2025 Q4(a)OR [3M: explain 3NF with example]

Third Normal Form (3NF): a table is in 3NF if (i) it is in **2NF** and (ii) it has **no transitive dependency** - no non-prime attribute depends on another non-prime attribute. (Formally: for every FD X → Y, either X is a super key or Y is a prime attribute.)

Example: EMPLOYEE(Emp_id, Emp_name, Dept_id, Dept_name). Emp_id → Dept_id and Dept_id → Dept_name, so Dept_name depends transitively on Emp_id → not in 3NF. **Decompose:** EMPLOYEE(Emp_id, Emp_name, Dept_id) and DEPARTMENT(Dept_id, Dept_name). Now every non-key attribute depends only on the key → 3NF.

Emp_id	Emp_name	Dept_id
1	Ravi	D1
2	Meera	D1
3	Amit	D2

Dept_id	Dept_name
D1	IT
D2	HR

Rule of thumb: 'every non-key attribute must depend on the key, the whole key and nothing but the key.'

4.4.4 Boyce-Codd Normal Form (BCNF) and 3NF vs BCNF

★ BCNF asked in W2025 Q5(a) [3M: differentiate 3NF vs BCNF]

Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF. A table is in BCNF if it is in 3NF and, for **every functional dependency X → Y, X is a super key**.

Example: TEACHING(Student, Subject, Teacher) with FDs: (Student, Subject) → Teacher and **Teacher → Subject** (each teacher teaches only one subject). Candidate keys: (Student, Subject) and (Student, Teacher). Teacher → Subject violates BCNF because Teacher is not a super key (but it is still in 3NF, since Subject is a prime attribute). **Decompose:** TEACHER_SUBJECT(Teacher, Subject) and STUDENT_TEACHER(Student, Teacher).

3NF vs BCNF

Point	3NF	BCNF
Condition	For X → Y: X is a super key **OR** Y is a prime attribute	For X → Y: X **must be** a super key
Strength	Weaker	Stronger (every BCNF table is in 3NF, not the reverse)
Redundancy	Some may remain	Almost completely removed
Dependency preservation	Always possible	Not always possible
Decomposition	Lossless and dependency preserving	Lossless, may lose some dependencies

Last-minute revision sheet - Unit 4

- Anomalies: insertion, update, deletion. Normalization removes redundancy and anomalies.
- FD: X → Y. Full FD: depends on whole key. Partial: part of key. Transitive: non-key → non-key.
- 1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency. 3NF: 2NF + no transitive dependency. BCNF: every determinant is a super key.
- Prime = part of a candidate key; non-prime = not part of any candidate key.
- 1NF techniques: one row per value; or separate table.
- Doctor_Patient key = (PatientID, DoctorID) → split into PATIENT, DOCTOR, PATIENT_DOCTOR.